

Capstone Project-I Report

TITLE

Secure & Transparent Online Voting System (SecureVote)

Submitted in partial fulfilment of the requirement for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE & ENGINEERING

Submitted by:

Student Name: _____

University Roll No.: _____

Under the Mentorship of

Mentor Name: _____

Designation: _____

Department of Computer Science and Engineering

Himachal Pradesh Technical University

Main Campus, at Daruhi, Hamirpur

(Session Year): 2025–2026

CANDIDATE'S DECLARATION

I hereby certify that the work which is being presented in the project report entitled “**Secure & Transparent Online Voting System (SecureVote)**” in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering** shall be carried out under the mentorship of **Mentor Name, Designation, Department of Computer Science and Engineering, Himachal Pradesh Technical University, Hamirpur.**

Name: _____

University Roll no: _____

Table of Contents

- **Chapter 1** Introduction
 - **Chapter 2** Literature Survey
 - **Chapter 3** Methodology
 - **Chapter 4** Result and Discussion
 - **Chapter 5** Conclusion and Future Work
 - References
-

Chapter 1

Introduction

Online elections and polls have become common in universities, societies, and organizations because they reduce manual effort, enable faster results, and allow participation from remote locations. However, traditional online voting solutions often struggle with key challenges such as:

- **Voter authentication** (preventing unauthorized access and impersonation)
- **One-person-one-vote enforcement**
- **Integrity and verification** (ensuring votes aren't altered)
- **Transparency** for stakeholders without compromising voter privacy
- **Operational monitoring** for administrators during voting

This capstone project, **SecureVote**, is a full-stack web-based voting system designed to provide secure authentication using OTP, real-time monitoring, and an immutable audit approach via **blockchain integration** (optional). The platform includes:

- A **Voter Portal** (frontend) for login, OTP verification, voting, vote history, and public dashboards.
- An **Admin Dashboard** (admin) for election creation, candidate management, voter imports, WhatsApp connectivity management, and monitoring.
- A **Backend API** (Express + MongoDB) that enforces business rules, integrates OTP delivery (WhatsApp/email/SMS), stores records, and supports local blockchain ledger plus optional on-chain verification using a smart contract.

Problem Statement

Design and implement a secure online voting platform that:

1. Authenticates voters strongly (OTP via WhatsApp as primary channel with fallback).
2. Ensures that each eligible voter can vote only once per election.
3. Provides transparency and auditability through a vote ledger (local chain) and optional smart-contract based verification.
4. Supports administrators with election management tools, bulk candidate/voter management, and real-time monitoring.

Objectives

- Build a scalable REST API for elections, voters, candidates, votes, and authentication.
- Implement OTP-based authentication with rate limiting and secure token handling.
- Implement vote casting with transactional consistency and audit trails.
- Provide real-time updates via Socket.io.
- Enable deployment-ready configuration (Render cloud deployment).

Scope

In scope:

- University/organization-level elections with defined start/end times
- Voter eligibility (roll list) validation
- OTP authentication using WhatsApp (Baileys) + email/SMS fallback
- Election management and monitoring
- Local blockchain ledger (MongoDB based)
- Optional Ethereum-compatible smart contract integration (via ethers.js)

Out of scope (future improvements):

- Large-scale national election requirements (biometrics, hardware, high assurance)
 - Full end-to-end verifiable cryptographic voting protocols (e.g., homomorphic tallying)
-

Chapter 2

Literature Survey

This chapter reviews existing work and commonly used approaches for online voting security and transparency.

OTP-based Authentication in Web Systems

OTP (One-Time Password) is widely used as a second-factor authentication approach. OTP improves security beyond passwords in cases where identity assurance is required. Delivery channels include:

- **SMS OTP** (common but vulnerable to SIM swap/phishing)
- **Email OTP** (depends on email security)
- **WhatsApp OTP** (can be more user-friendly; relies on WhatsApp account security)

SecureVote adopts WhatsApp OTP as the primary channel with **fallback** to SMS/email to ensure reliability.

Role-based Admin Portals

Most voting applications separate voter and admin features. Admin portals typically include:

- Election scheduling and status management
- Candidate registration
- Monitoring and reporting

SecureVote implements a role-protected admin panel with JWT-based authorization.

Blockchain for Auditability

Blockchain is frequently proposed to improve transparency and integrity in voting. Common patterns:

1. **Record vote hashes on-chain** to prevent tampering.
2. Maintain a **local append-only ledger** (hash chain) when on-chain cost/complexity is not desired.

SecureVote uses a hybrid concept:

- **Local blockchain ledger** stored in MongoDB for continuous audit trail.
- **Optional smart contract integration** so votes can be verified against chain transactions.

Real-time Monitoring in Modern Web Apps

Real-time dashboards are useful during elections for monitoring turnout and detecting anomalies. WebSockets (Socket.io) are a common solution.

SecureVote broadcasts vote updates and vote counts in real-time to connected dashboards.

Chapter 3

Methodology

This chapter describes the overall methodology, system design, and implementation approach.

System Architecture

The system follows a **three-tier architecture**.

1. Presentation Layer

- Voter Application (frontend/) – React + Vite
- Admin Dashboard (admin/) – React + Vite

2. Application Layer

- Backend API (backend/) – Node.js + Express
- OTP services, election scheduling, Socket.io communications

3. Data Layer

- MongoDB database (voters, elections, votes, candidates, students)
- Local blockchain ledger in MongoDB
- Optional public blockchain via smart contract

High-level Flow

```
flowchart TD
    A[Voter enters Roll/Aadhaar + Name + Contact] --> B[Request OTP API]
    B --> C[OTP Service]
    C -->|Primary| D[WhatsApp via Baileys]
    C -->|Fallback| E[Email (SMTP/Ethereal) / SMS (Twilio)]
    A --> F[Verify OTP API]
    F --> G[JWT Token Issued]
    G --> H[Fetch ongoing elections]
    H --> I[Cast Vote]
    I --> J[MongoDB Transaction]
    J --> K[Update Candidate voteCount]
    J --> L[Create Vote record + hash]
    L --> M[Local Blockchain block (optional)]
    L --> N[On-chain tx link/verify (optional)]
    J --> O[Append voter history]
    O --> P[Emit real-time socket updates]
```

Key Modules

1) Authentication and OTP

- backend/routes/voterRoutes.js implements:
 - POST /api/voter/request-otp
 - POST /api/voter/verify-otp

- backend/config/otpService.js generates OTP, stores it in memory, and triggers delivery.
- backend/config/whatsappService.js uses Baileys multi-file auth state in backend/baileys_auth to reconnect to the last linked WhatsApp account.

Security measures:

- Rate limiting on OTP endpoints (/api/voter/request-otp and /api/voter/verify-otp).
- Hashing of voter identifier for storage (sha256) to avoid storing raw Aadhaar in DB.
- JWT-based authentication for voter sessions.

2) Voting Process

backend/routes/voteRoutes.js implements POST /api/vote/cast with:

- Eligibility checks (ongoing election, time window)
- One-vote-per-election enforcement (checks voter history)
- Atomic updates using a MongoDB session/transaction
- Vote hash generation for receipt
- Real-time event emission via Socket.io

3) Transparency and Blockchain

- **Local chain:** a block is computed and stored in MongoDB (hash-chaining).
- **On-chain option:** backend/services/web3Service.js uses ethers.js to connect to a contract and verify transaction receipts.

4) Admin Operations

Admin dashboard features include:

- Election and candidate management
- Bulk import of voters/students (Excel/CSV)
- WhatsApp connectivity checks and QR pairing from admin panel

Technology Stack

- **Frontend/Admin:** React, Vite, Tailwind CSS, Axios, Socket.io client
- **Backend:** Node.js, Express, Mongoose, Socket.io, Helmet, CORS, Rate limiting
- **OTP/Communication:** Baileys (WhatsApp), Nodemailer (email), Twilio (SMS optional)
- **Blockchain:** ethers.js + Solidity smart contract (optional)
- **Deployment:** Render (backend as web service; frontends as static sites or served by backend depending on configuration)

Chapter 4

Result and Discussion

Functional Results

SecureVote successfully implements:

1. OTP-based voter login

- Voters can request OTP via WhatsApp.
- On failure/unavailability, OTP delivery falls back to email/SMS or console mock (development mode).

2. Election lifecycle

- Elections are scheduled with start and end times.
- Backend scheduler updates election status automatically.

3. Vote casting with integrity controls

- Vote casting is blocked if:
 - election is not ongoing
 - voter already voted in the same election
- Vote receipts are generated (voteHash).

4. Real-time monitoring

- Socket.io events broadcast vote updates to dashboards.

5. Transparency mechanisms

- Local blockchain ledger creates an immutable chain of vote records.
- Optional on-chain verification validates that a txHash really exists and is confirmed.

Discussion: Security and Reliability

• Strengths

- OTP authentication reduces password-related risks.
- Hashing of identifiers improves privacy.
- Multi-layer audit (DB + local chain + optional on-chain) provides stronger integrity.
- Rate limiting reduces OTP brute-force attacks.

• Limitations

- OTP store is in-memory by default; scaling horizontally would require Redis/shared storage.
- Baileys WhatsApp session persistence depends on server storage; cloud hosting may require persistent disks.
- Full cryptographic secrecy and coercion resistance is not implemented (requires advanced protocols).

Chapter 5

Conclusion and Future Work

Conclusion

SecureVote demonstrates a practical online voting system that balances usability and security. The system provides OTP-based authentication, strong enforcement of voting constraints, real-time monitoring, and audit-friendly vote storage. The hybrid blockchain approach offers transparency while allowing deployments that do not require mandatory on-chain operations.

Future Work

- Replace in-memory OTP store with Redis for scalability.
 - Add persistent storage for WhatsApp auth state on cloud deployments.
 - Enhance security with device binding and anomaly detection.
 - Add advanced cryptographic voting features (end-to-end verifiable schemes).
 - Improve accessibility and multilingual support.
-

References

- [1] N. K. Kanhere and S. T. Birchfield, "Real-time incremental segmentation and tracking of vehicles at low camera angles using stable features," *IEEE Trans. Intell. Transp. Syst.*, vol. 9, no. 1, pp. 148–160, March 2008.
- [2] K. Onoguchi, "Moving object detection using a cross correlation between a short accumulated histogram and a long accumulated histogram," in *Proc. 18th Int. Conf. on Pattern Recognition*, Hong Kong, Aug. 20–24, 2006, vol. 4, pp. 896–899.
- [3] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, 2nd ed. MIT Press, 2001.
- [4] Express.js Documentation. <https://expressjs.com/>
- [5] MongoDB Documentation. <https://www.mongodb.com/docs/>
- [6] Socket.io Documentation. <https://socket.io/docs/>
- [7] Ethers.js Documentation. <https://docs.ethers.org/>
- [8] Baileys (WhatsApp Web API) Documentation. <https://github.com/WhiskeySockets/Baileys/>